

TRANSPORTATION & FLEET OPERATIONS ANALYTICS DASHBOARD

Complete Project Documentation

Platform: Microsoft Power BI, Excel, SQL, Python

Core technologies: Power Query, DAX, Data Modeling, SQL, Python (pandas, matplotlib), Excel/CSV

Project domain: Transportation, Fleet Operations, Cost and Revenue Analytics

JAGADEESHKUMAR JALLA
BNO10

Document Contents

- 1. Executive Summary
- 2. Business Problem and Objectives
- 3. Project Scope and Questions
- 4. Tools and Technologies
- 5. Dataset Overview and Data Dictionary
- 6. Data Cleaning and Transformation
- 7. Data Model and Relationships
- 8. DAX Measures and Calculated Columns
- 9. Dashboard Design and Visual Specifications
- 10. SQL Analysis (new)
- 11. Python Analysis (new)
- 12. Analysis Workflow and Validation
- 13. Business Insights and Recommendations Framework
- 14. Testing, Limitations, and Future Enhancements
- 15. Project Conclusion
- 16. Resume Description and Interview Explanation

1. Executive Summary

The Transportation & Fleet Operations Analytics Dashboard is a business intelligence project built with Microsoft Power BI. It brings trip, vehicle, driver, route, fuel, maintenance, revenue, and operating-cost information into an interactive report. The dashboard is designed to help users monitor operational KPIs, compare performance across vehicle types and routes, and investigate trends over time.

The project is supported by two independent analysis layers. **SQL** is used to profile the data, run data-quality checks and compute KPIs and rankings directly from the trip table. **Python** (pandas, matplotlib) is used for exploratory analysis, outlier detection, correlation analysis and chart generation. Results from SQL and Python are reconciled against the DAX measures in Power BI so that every number in the dashboard can be verified.

2. Business Problem and Objectives

Transportation operations generate data across trips, fleet assets, drivers, fuel usage, maintenance, and revenue. When these details are reviewed separately, it can be difficult to understand cost drivers or compare operational performance consistently.

Objectives

- Track total trips, distance travelled, revenue, operating cost, and profit.

- Compare operating costs, fuel consumption, and fuel efficiency across vehicle types.
- Monitor trip completion rate and trip-status distribution.
- Review driver and customer ratings as service-quality indicators.
- Analyze maintenance expenses by maintenance type, vehicle, and time period.
- Enable interactive exploration using date, vehicle, driver, route, and location filters.
- Cross-validate dashboard KPIs using independent SQL queries and a Python (pandas) analysis.

3. Project Scope and Business Questions

The project focuses on descriptive analytics: what happened, how performance differs by category, and how metrics change over time. It does not automatically establish causation or forecast future performance.

Questions the dashboard should answer

- How many trips were recorded, and how far did vehicles travel?
- How do revenue and operating costs change month by month?
- Which vehicle types contribute the most trips, revenue, or operating cost?
- How much fuel was consumed, and what is the average distance per litre?
- What proportion of trips is completed?
- How do maintenance costs vary by maintenance type or vehicle?
- How are distance, duration, revenue, and operating cost related?
- How do driver ratings and customer ratings vary across the fleet?

4. Tools and Technologies

Tool / Technology	Purpose
Microsoft Power BI Desktop	Create data model, measures, visuals, filters, and report pages.
Power Query	Inspect, clean, transform, and combine source data.
DAX	Calculate KPIs, ratios, completion rates, and analytical measures.
Star-schema data modeling	Organize the fact table and descriptive dimensions for reliable filtering.
SQL (PostgreSQL syntax)	Data profiling, data-quality checks, KPI aggregation, ranking and window-function analysis directly on the trip table. Notes for MySQL / SQL Server are given in Section 10.
Python 3 (pandas, NumPy, matplotlib, seaborn)	Exploratory data analysis, outlier detection, correlation analysis, statistical summaries and chart generation.
Jupyter Notebook / VS Code	Interactive environment for running and documenting the Python analysis.
Excel / CSV	Provide the raw source dataset, if applicable.

5. Dataset Overview and Data Dictionary

The project dataset discussed contains trip-level records with 27 fields. Confirm the exact field names and data types in Power Query before implementing the model.

Field	Description / Suggested type
Trip_ID	Unique trip identifier; text or whole number.

Field	Description / Suggested type
Trip_Date	Date of trip; Date type.
Vehicle_ID	Vehicle identifier; key/category.
Vehicle_Type	Vehicle category; text.
Driver_ID	Driver identifier; key/category.
Driver_Name	Driver display name; text.
Driver_Rating	Driver rating; decimal number.
Route_ID	Route identifier; key/category.
Source / Destination	Trip origin and destination; text.
Distance_KM	Distance travelled; decimal number, kilometres.
Duration_Minutes	Trip duration; decimal/whole number, minutes.
Fuel_Consumed_Liters	Fuel consumed; decimal number, litres.
Fuel_Cost_INR	Fuel cost; currency/decimal, INR.
Maintenance_Cost_INR	Maintenance expense; currency/decimal, INR.
Toll_Cost_INR	Toll expense; currency/decimal, INR.
Driver_Cost_INR	Driver expense; currency/decimal, INR.
Operating_Cost_INR	Total operating cost as supplied; currency/decimal, INR.
Revenue_INR	Trip revenue; currency/decimal, INR.
Payment_Mode	Payment category; text.
Trip_Status	Status such as Completed; text.
Customer_Rating	Customer rating; decimal number.
Maintenance_Type	Maintenance category; text.
Location	Location associated with the record; text.
Vehicle_Model	Vehicle model; text.
Fuel_Type	Fuel category; text.
Vehicle_Age_Years	Vehicle age; whole/decimal number, years.

6. Data Cleaning and Transformation

Recommended Power Query steps

- Import the source Excel or CSV file into Power BI.
- Promote the correct row to headers and remove fully blank rows.
- Set data types: dates as Date, IDs as text or whole number consistently, and numeric measures as decimal/whole numbers.
- Trim and clean text columns; standardize spelling and category labels.
- Check duplicate Trip_ID values and determine whether duplicates are valid repeated records or data errors.
- Review nulls in key fields and decide whether to correct, retain, or exclude records based on business rules.

- Check outliers such as extremely high distance, duration, fuel usage, or costs; investigate rather than automatically deleting them.
- Create a date table and dimension tables where useful. Ensure every dimension key maps correctly to the fact table.
- Apply transformations and verify row counts before and after cleaning.

Important cost validation

Confirm whether Operating_Cost_INR already includes fuel, maintenance, toll, and driver costs. If it does, do not add those components to operating cost again. If it does not, define the total cost calculation clearly and consistently. Query 10.2 (SQL) and Step 11.4 (Python) test this directly.

7. Data Model and Relationships

Use a star schema: one central Fact_Trips table containing trip-level numeric values and keys, with dimension tables providing descriptive attributes. Create one-to-many relationships from each dimension to Fact_Trips, with single-direction filtering unless a justified scenario requires otherwise.

Table	Example contents	Relationship
Fact_Trips	Trip_ID, DateKey, VehicleKey, DriverKey, RouteKey, distance, duration, fuel, costs, revenue, status/rating fields as appropriate.	Central fact table.
Dim_Date	DateKey, Date, Year, Quarter, Month_Number, Month_Name, Month_Year, Day_Name.	Dim_Date[DateKey] 1 → * Fact_Trips[DateKey].
Dim_Vehicle	VehicleKey, Vehicle_ID, Vehicle_Type, Vehicle_Model, Fuel_Type, Vehicle_Age_Years.	Dim_Vehicle[VehicleKey] 1 → * Fact_Trips[VehicleKey].
Dim_Driver	DriverKey, Driver_ID, Driver_Name.	Dim_Driver[DriverKey] 1 → * Fact_Trips[DriverKey].
Dim_Route	RouteKey, Route_ID, Source, Destination, Route_Name.	Dim_Route[RouteKey] 1 → * Fact_Trips[RouteKey].
Optional dimensions	Dim_Maintenance, Dim_Payment, Dim_TripStatus, Dim_Location when useful.	Each dimension key should map to the corresponding fact key.

Modeling notes

- Do not create separate index columns in fact and dimension tables and assume they match. Build distinct dimension keys and merge/map them back to the fact table.
- Mark Dim_Date as the date table and sort Month_Name by Month_Number.
- Use dimension fields for slicers and chart axes where possible.
- Keep Trip_ID unique at the trip grain, or document the actual grain if multiple rows can exist per trip.

8. DAX Measures and Calculated Columns

Create these measures after confirming table and column names. If your model uses different names, adjust the formulas. Format currency measures as INR, percentages as Percentage, distance as kilometres, and fuel as litres.

Total Trips

```
Total Trips = DISTINCTCOUNT(Fact_Trips[Trip_ID])
```

Total Revenue

```
Total Revenue = SUM(Fact_Trips[Revenue_INR])
```

Total Operating Cost

Total Operating Cost = SUM(Fact_Trips[Operating_Cost_INR])

Total Profit

Total Profit = [Total Revenue] - [Total Operating Cost]

Profit Margin %

Profit Margin % = DIVIDE([Total Profit], [Total Revenue], 0)

Total Distance

Total Distance = SUM(Fact_Trips[Distance_KM])

Total Fuel Consumed

Total Fuel Consumed = SUM(Fact_Trips[Fuel_Consumed_Liters])

Fuel Efficiency KMPL

Fuel Efficiency KMPL = DIVIDE([Total Distance], [Total Fuel Consumed], 0)

Completed Trips

Completed Trips = CALCULATE(DISTINCTCOUNT(Fact_Trips[Trip_ID]), Fact_Trips[Trip_Status] = "Completed")

Trip Completion Rate %

Trip Completion Rate % = DIVIDE([Completed Trips], [Total Trips], 0)

Average Trip Distance

Average Trip Distance = AVERAGE(Fact_Trips[Distance_KM])

Average Trip Duration

Average Trip Duration = AVERAGE(Fact_Trips[Duration_Minutes])

Average Driver Rating

Average Driver Rating = AVERAGE(Fact_Trips[Driver_Rating])

Average Customer Rating

Average Customer Rating = AVERAGE(Fact_Trips[Customer_Rating])

Average Revenue per Trip

Average Revenue per Trip = DIVIDE([Total Revenue], [Total Trips], 0)

Average Operating Cost per Trip

Average Operating Cost per Trip = DIVIDE([Total Operating Cost], [Total Trips], 0)

Total Fuel Cost

Total Fuel Cost = SUM(Fact_Trips[Fuel_Cost_INR])

Total Maintenance Cost

Total Maintenance Cost = SUM(Fact_Trips[Maintenance_Cost_INR])

Total Toll Cost

Total Toll Cost = SUM(Fact_Trips[Toll_Cost_INR])

Total Driver Cost

Total Driver Cost = SUM(Fact_Trips[Driver_Cost_INR])

Fuel Cost %

Fuel Cost % = DIVIDE([Total Fuel Cost], [Total Operating Cost], 0)

Maintenance Cost %

Maintenance Cost % = DIVIDE([Total Maintenance Cost], [Total Operating Cost], 0)

Revenue per KM

Revenue per KM = DIVIDE([Total Revenue], [Total Distance], 0)

Operating Cost per KM

Operating Cost per KM = DIVIDE([Total Operating Cost], [Total Distance], 0)

Optional calculated columns

Use calculated columns only when a row-level result is required for grouping, filtering, or detailed analysis. Measures are usually preferred for totals and ratios.

Profit_INR

Profit_INR = Fact_Trips[Revenue_INR] - Fact_Trips[Operating_Cost_INR]

Cost_Per_KM

Cost_Per_KM = DIVIDE(Fact_Trips[Operating_Cost_INR], Fact_Trips[Distance_KM], 0)

Revenue_Per_KM

Revenue_Per_KM = DIVIDE(Fact_Trips[Revenue_INR], Fact_Trips[Distance_KM], 0)

Duration_Hours

Duration_Hours = DIVIDE(Fact_Trips[Duration_Minutes], 60, 0)

Profit_Status

Profit_Status = IF(Fact_Trips[Revenue_INR] >= Fact_Trips[Operating_Cost_INR], "Non-negative", "Loss")

9. Dashboard Design and Visual Specifications

Recommended layout: a clean 16:9 report canvas with a consistent theme, readable labels, aligned visuals, and slicers grouped in a dedicated area.

Suggested report pages

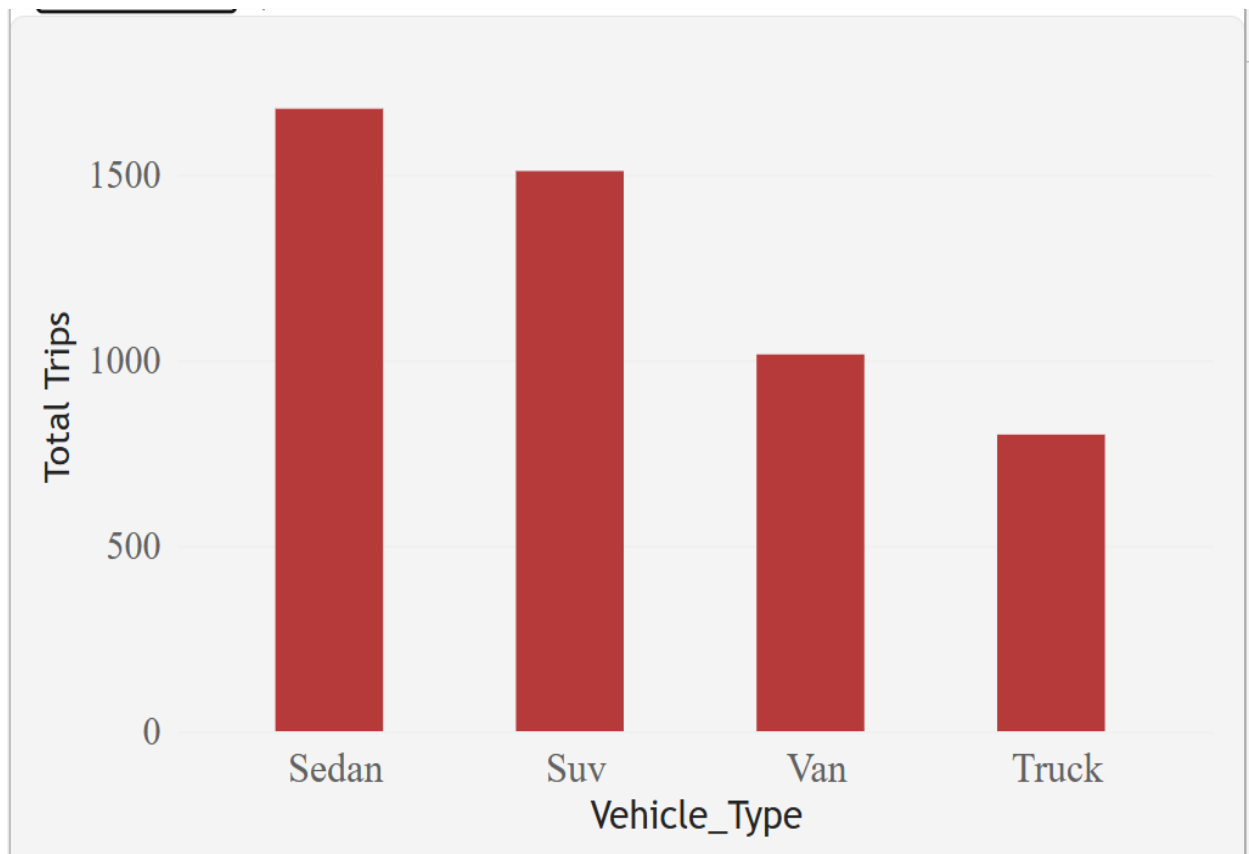
Page	Purpose	Suggested visuals
1. Executive Overview	Quick operational summary.	KPI cards: Total Trips, Total Revenue, Total Operating Cost, Total Profit; monthly Revenue vs Cost line chart; trip status donut; vehicle type comparison.
2. Fleet Performance	Compare vehicles and vehicle types.	Trips by Vehicle Type; Fuel Consumed by Vehicle Type; Fuel Efficiency KMPL; Vehicle Age vs Maintenance Cost scatter; vehicle/location slicers.
3. Route & Trip Analysis	Understand route and trip patterns.	Profit by Route; Distance vs Revenue scatter; Distance vs Duration scatter; monthly distance/fuel trend; Source/Destination filters.
4. Cost & Maintenance	Investigate expenses.	Operating Cost by Vehicle Type; Maintenance Cost by Type; cost composition stacked bar/area; monthly fuel cost trend; decomposition tree if useful.
5. Driver & Customer	Monitor service indicators.	Average Driver Rating by Driver; Average Customer Rating by Vehicle Type; Trip Completion Rate trend; rating distribution.

Visual field mapping

- **Line chart — Monthly Revenue vs Operating Cost:** X-axis Dim_Date[Month_Year]; values [Total Revenue] and [Total Operating Cost]. Sort Month_Year chronologically.
- **Clustered column — Total Trips by Vehicle Type:** X-axis Dim_Vehicle[Vehicle_Type]; Y-axis [Total Trips].
- **Stacked column — Revenue by Month and Vehicle Type:** X-axis Dim_Date[Month_Year]; legend Dim_Vehicle[Vehicle_Type]; values [Total Revenue].
- **Scatter — Distance vs Revenue:** X Fact_Trips[Distance_KM]; Y Fact_Trips[Revenue_INR]; Details Trip_ID if the visual supports it.
- **Scatter — Vehicle Age vs Maintenance Cost:** X Dim_Vehicle[Vehicle_Age_Years]; Y [Total Maintenance Cost]; Details Vehicle_ID if available.
- **Donut — Trip Status Distribution:** legend Trip_Status; values [Total Trips].
- **Pie — Revenue by Fuel Type:** legend Dim_Vehicle[Fuel_Type]; values [Total Revenue]. Keep categories limited.
- **Map — Trips by Location:** location field Location; bubble size [Total Trips]. Add country/state context or coordinates if names are ambiguous.
- **Stacked area/bar cost composition:** create a normalized cost-type table and a measure that returns the selected component. Do not simply place unrelated cost measures into a field intended for one shared category.

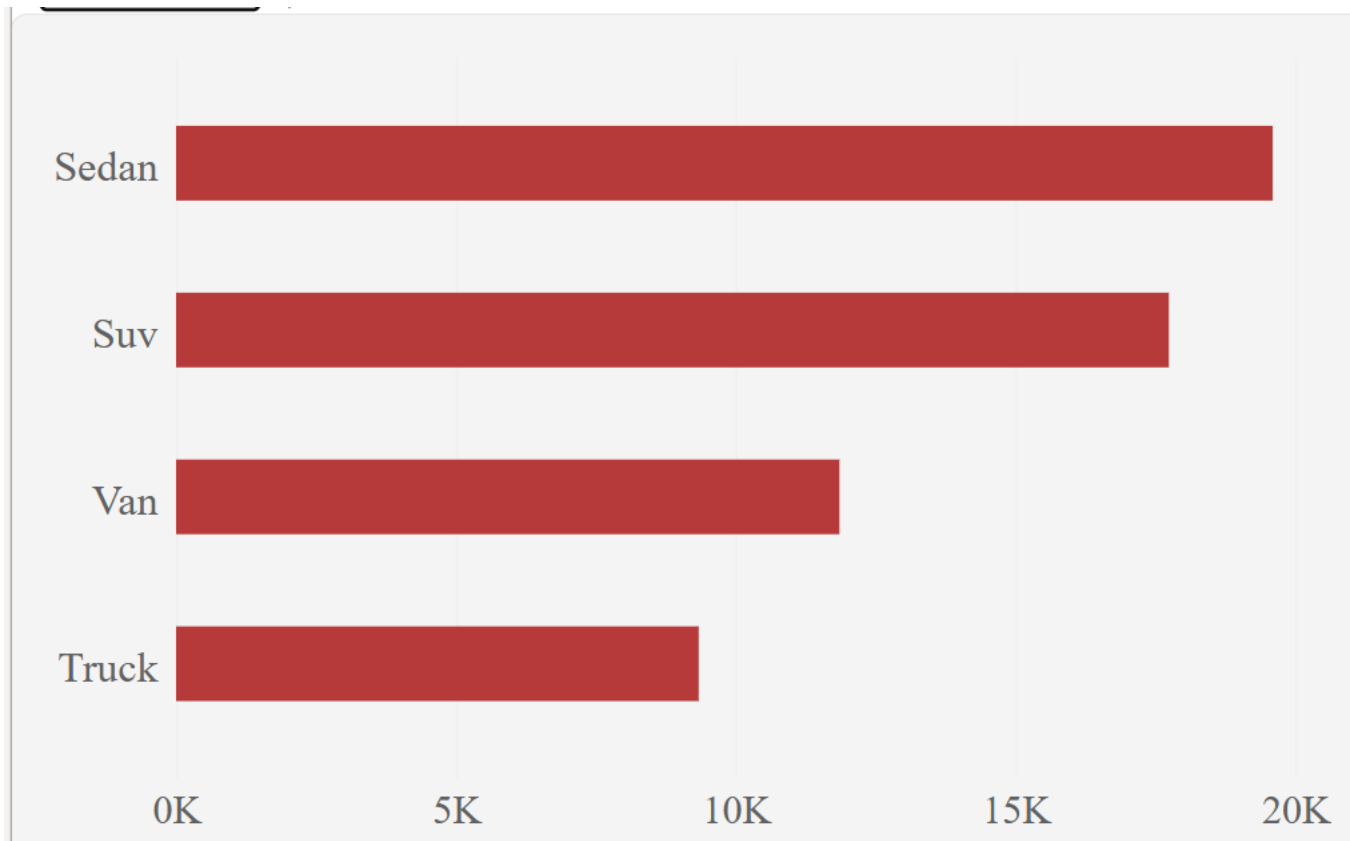
Total trips by vehicle type

- Visual: Clustered column chart
- X-axis: Vehicle_Type
- Y-axis: Total Trips



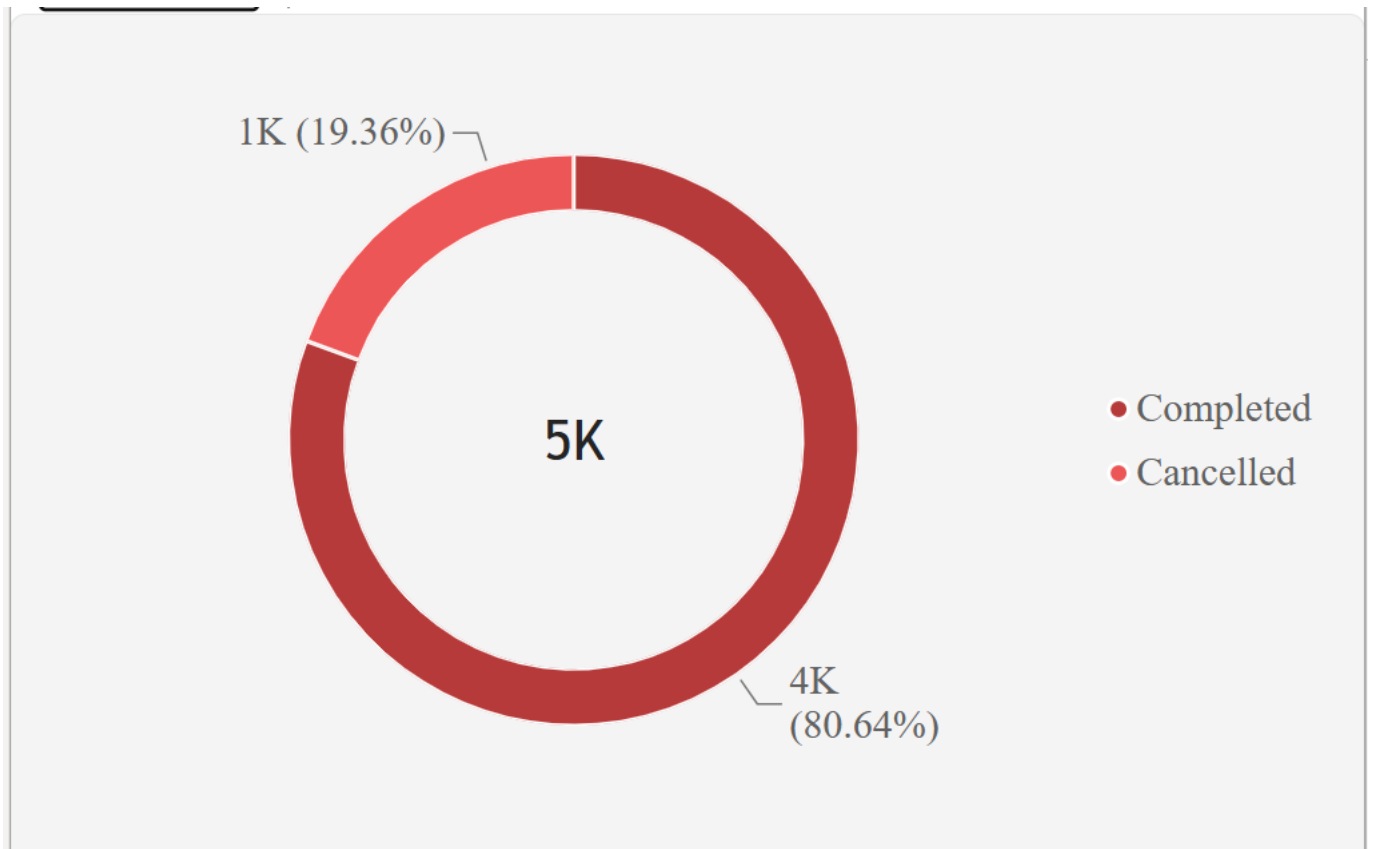
Sum of fuel efficiency kmpl by vehicle type

- Visual: Clustered bar chart
- Y-axis: Vehicle_Type
- X-axis: Sum of Fuel Efficiency_KMPL



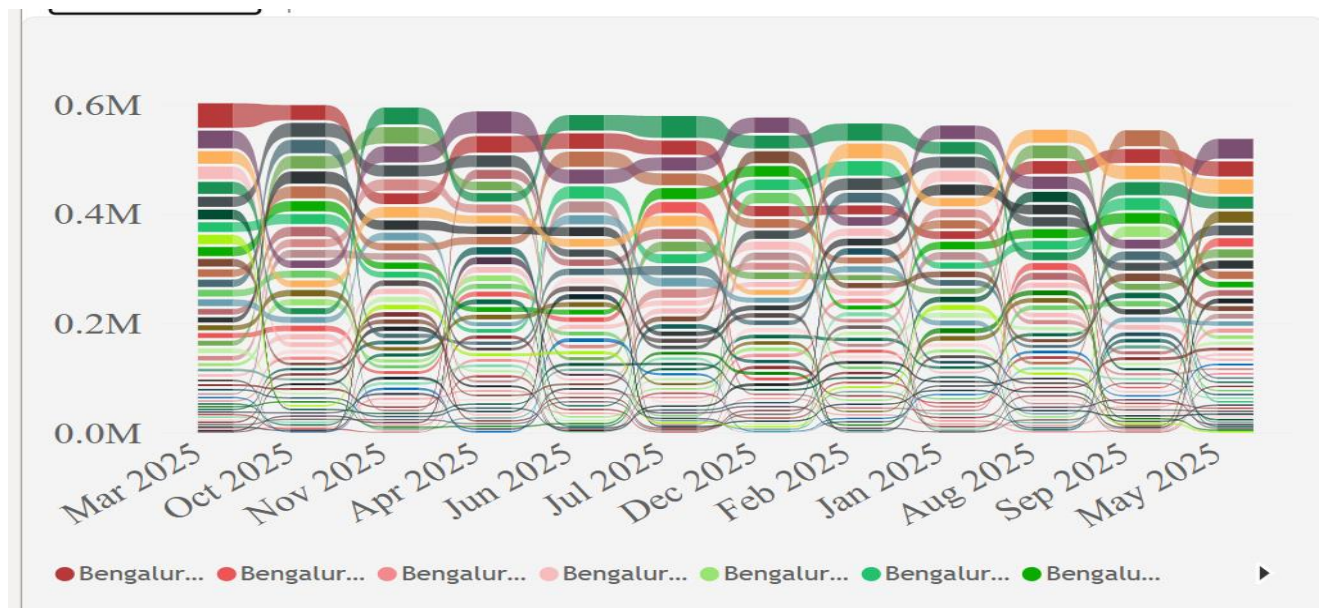
Total trips by Trip status

- Visual: Donut chart
- Legend: Trip_Status
- Values: Total Trips



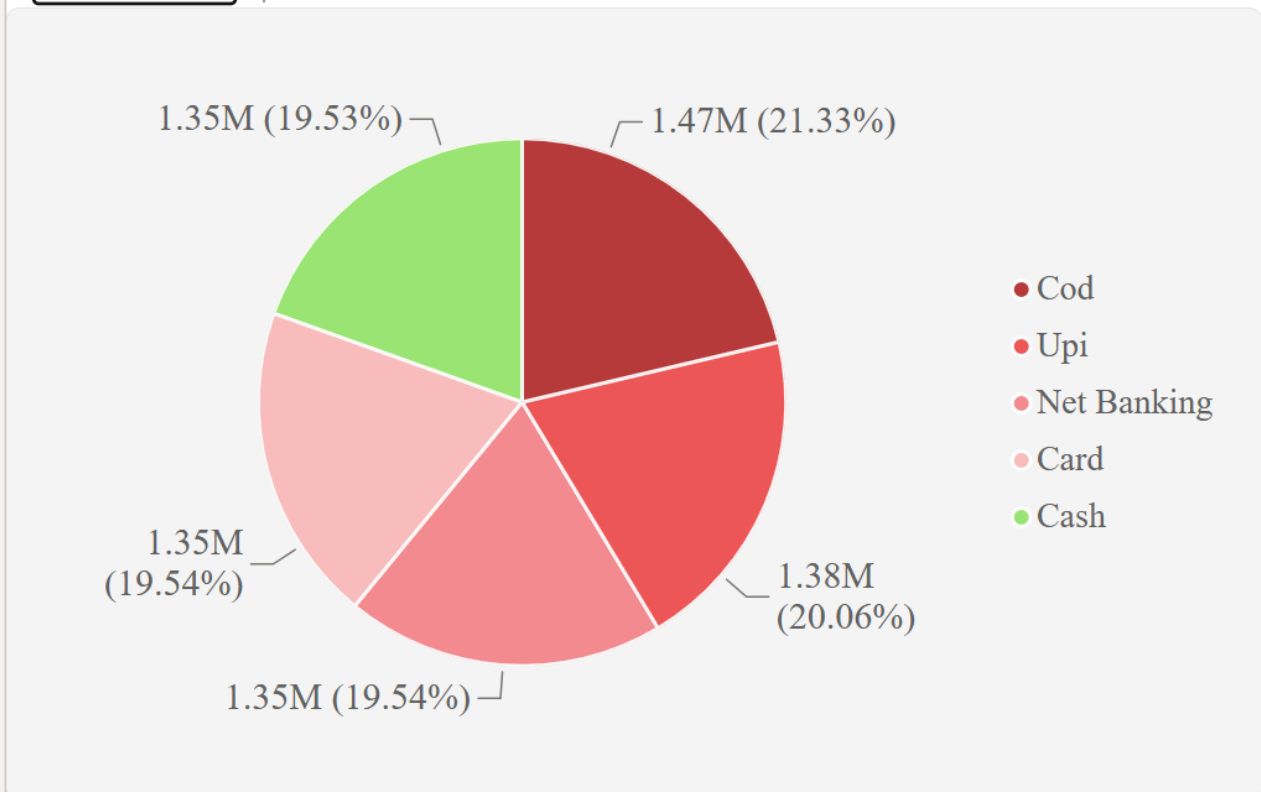
Total revenue by month year and route name

- Visual: Ribbon chart
- X-axis: Month_Year
- Y-axis: Total Revenue
- Legend: Route_Name



Total revenue by payment mode

- Visual: Pie chart
- Legend: Payment_Mode
- Values: Total Revenue



10. SQL Analysis

SQL is used as an independent analysis layer. It answers the same business questions as the dashboard directly from the trip table, and it provides reference numbers against which the DAX measures are reconciled. The queries below use **PostgreSQL** syntax; adaptation notes for MySQL and SQL Server are at the end of this section.

10.1 Table setup

Load the cleaned CSV into a single trip-level table. Column names follow the data dictionary in Section 5.

```
CREATE TABLE fleet_trips (  
    trip_id          VARCHAR(20) PRIMARY KEY,  
    trip_date        DATE          NOT NULL,  
    vehicle_id       VARCHAR(20)  NOT NULL,  
    vehicle_type     VARCHAR(50),  
    driver_id        VARCHAR(20),  
    driver_name       VARCHAR(100),  
    driver_rating     NUMERIC(3,2),  
    route_id         VARCHAR(20),  
    source           VARCHAR(100),  
    destination      VARCHAR(100),  
    distance_km      NUMERIC(10,2),  
    duration_minutes NUMERIC(10,2),  
    fuel_consumed_liters NUMERIC(10,2),  
    fuel_cost_inr     NUMERIC(12,2),  
    maintenance_cost_inr NUMERIC(12,2),  
    toll_cost_inr     NUMERIC(12,2),  
    driver_cost_inr   NUMERIC(12,2),  
    operating_cost_inr NUMERIC(12,2),  
    revenue_inr       NUMERIC(12,2),  
    payment_mode      VARCHAR(30),  
    trip_status       VARCHAR(30),  
    customer_rating   NUMERIC(3,2),  
    maintenance_type  VARCHAR(50),  
    location          VARCHAR(100),  
    vehicle_model     VARCHAR(50),  
    fuel_type         VARCHAR(30),  
    vehicle_age_years NUMERIC(5,1)  
);  
  
-- Load from CSV (psql); adjust the path and header option to your file  
\copy fleet_trips FROM 'fleet_trips.csv' WITH (FORMAT csv, HEADER true);
```

If Trip_ID is not truly unique in your data, remove the PRIMARY KEY constraint, load the data, and use Query 10.2 to investigate the duplicates before deciding how to treat them.

10.2 Data quality checks

Row count, date range and duplicate Trip_ID

```
SELECT COUNT(*)           AS total_rows,  
       COUNT(DISTINCT trip_id) AS distinct_trips,  
       MIN(trip_date)      AS first_date,  
       MAX(trip_date)      AS last_date  
FROM fleet_trips;  
  
SELECT trip_id, COUNT(*) AS occurrences  
FROM fleet_trips  
GROUP BY trip_id  
HAVING COUNT(*) > 1;
```

Missing values in key fields

```
SELECT
    COUNT(*) FILTER (WHERE trip_date IS NULL)           AS null_trip_date,
    COUNT(*) FILTER (WHERE vehicle_id IS NULL)          AS null_vehicle_id,
    COUNT(*) FILTER (WHERE distance_km IS NULL)         AS null_distance,
    COUNT(*) FILTER (WHERE revenue_inr IS NULL)         AS null_revenue,
    COUNT(*) FILTER (WHERE operating_cost_inr IS NULL)  AS null_operating_cost,
    COUNT(*) FILTER (WHERE trip_status IS NULL)         AS null_trip_status,
    COUNT(*) FILTER (WHERE driver_rating IS NULL)       AS null_driver_rating,
    COUNT(*) FILTER (WHERE customer_rating IS NULL)     AS null_customer_rating
FROM fleet_trips;
```

Invalid or implausible values

```
SELECT trip_id, distance_km, duration_minutes, fuel_consumed_liters,
       operating_cost_inr, revenue_inr, driver_rating, customer_rating
FROM fleet_trips
WHERE distance_km < 0 OR duration_minutes < 0 OR fuel_consumed_liters < 0
   OR operating_cost_inr < 0 OR revenue_inr < 0
   OR driver_rating NOT BETWEEN 0 AND 5      -- adjust to your rating scale
   OR customer_rating NOT BETWEEN 0 AND 5;
```

Category spelling consistency

```
SELECT trip_status, COUNT(*) FROM fleet_trips GROUP BY trip_status ORDER BY 2 DESC;
SELECT vehicle_type, COUNT(*) FROM fleet_trips GROUP BY vehicle_type ORDER BY 2 DESC;
SELECT fuel_type, COUNT(*) FROM fleet_trips GROUP BY fuel_type ORDER BY 2 DESC;
SELECT payment_mode, COUNT(*) FROM fleet_trips GROUP BY payment_mode ORDER BY 2 DESC;
```

Does Operating_Cost_INR already include its components?

This query tests the assumption flagged in Sections 6 and 8. It compares the supplied operating cost with the sum of the four cost components.

```
SELECT
    COUNT(*)                                           AS trips_checked,
    COUNT(*) FILTER (WHERE ABS(operating_cost_inr -
        (fuel_cost_inr + maintenance_cost_inr + toll_cost_inr + driver_cost_inr)) < 1)
                                                AS operating_cost_equals_sum,
    COUNT(*) FILTER (WHERE operating_cost_inr >
        (fuel_cost_inr + maintenance_cost_inr + toll_cost_inr + driver_cost_inr) + 1)
                                                AS operating_cost_higher,
    COUNT(*) FILTER (WHERE operating_cost_inr <
        (fuel_cost_inr + maintenance_cost_inr + toll_cost_inr + driver_cost_inr) - 1)
                                                AS operating_cost_lower
FROM fleet_trips;
```

If most rows fall in the first bucket, the field is the sum of its components and must not be added to them again. If not, document the definition you adopt.

10.3 Overall KPIs

These values should match the Power BI KPI cards when no filters are applied.

```
SELECT
    COUNT(DISTINCT trip_id)           AS total_trips,
    SUM(revenue_inr)                   AS total_revenue,
    SUM(operating_cost_inr)           AS total_operating_cost,
    SUM(revenue_inr) - SUM(operating_cost_inr) AS total_profit,
    ROUND(100.0 * (SUM(revenue_inr) - SUM(operating_cost_inr))
        / NULLIF(SUM(revenue_inr), 0), 2) AS profit_margin_pct,
    SUM(distance_km)                  AS total_distance_km,
    SUM(fuel_consumed_liters)         AS total_fuel_liters,
    ROUND(SUM(distance_km) / NULLIF(SUM(fuel_consumed_liters), 0), 2) AS fuel_efficiency_kmpl,
    ROUND(100.0 * COUNT(*) FILTER (WHERE trip_status = 'Completed')
        / NULLIF(COUNT(*), 0), 2) AS completion_rate_pct,
    ROUND(AVG(driver_rating), 2)      AS avg_driver_rating,
    ROUND(AVG(customer_rating), 2)    AS avg_customer_rating
FROM fleet_trips;
```

10.4 Monthly trend with window functions

Monthly revenue, cost and profit, with month-over-month change and a three-month moving average of profit.

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', trip_date)::date AS month_start,
           COUNT(*) AS trips,
           SUM(revenue_inr) AS revenue,
           SUM(operating_cost_inr) AS operating_cost,
           SUM(revenue_inr - operating_cost_inr) AS profit
    FROM fleet_trips
    GROUP BY 1
)
SELECT month_start, trips, revenue, operating_cost, profit,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month_start))
             / NULLIF(LAG(revenue) OVER (ORDER BY month_start), 0), 2) AS revenue_mom_pct,
       ROUND(AVG(profit) OVER (ORDER BY month_start
                              ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS profit_3m_avg
FROM monthly
ORDER BY month_start;
```

10.5 Vehicle type and fleet performance

```
SELECT vehicle_type,
       COUNT(*) AS trips,
       SUM(distance_km) AS distance_km,
       SUM(fuel_consumed_liters) AS fuel_liters,
       ROUND(SUM(distance_km) / NULLIF(SUM(fuel_consumed_liters), 0), 2) AS kmpl,
       SUM(revenue_inr) AS revenue,
       SUM(operating_cost_inr) AS operating_cost,
       ROUND(SUM(operating_cost_inr) / NULLIF(SUM(distance_km), 0), 2) AS cost_per_km,
       ROUND(SUM(revenue_inr) / NULLIF(SUM(distance_km), 0), 2) AS revenue_per_km,
       ROUND(100.0 * SUM(revenue_inr - operating_cost_inr)
             / NULLIF(SUM(revenue_inr), 0), 2) AS profit_margin_pct
FROM fleet_trips
GROUP BY vehicle_type
ORDER BY revenue DESC;
```

Fuel efficiency and maintenance by fuel type and vehicle age band

```
SELECT fuel_type,
       CASE WHEN vehicle_age_years < 3 THEN '0-2 yrs'
            WHEN vehicle_age_years < 6 THEN '3-5 yrs'
            WHEN vehicle_age_years < 9 THEN '6-8 yrs'
            ELSE '9+ yrs' END AS age_band,
       COUNT(*) AS trips,
       ROUND(SUM(distance_km) / NULLIF(SUM(fuel_consumed_liters), 0), 2) AS kmpl,
       ROUND(AVG(maintenance_cost_inr), 2) AS avg_maintenance_cost
FROM fleet_trips
GROUP BY fuel_type, age_band
ORDER BY fuel_type, age_band;
```

10.6 Route profitability and ranking

```
WITH route_stats AS (
    SELECT route_id, source, destination,
           COUNT(*) AS trips,
           SUM(revenue_inr) AS revenue,
           SUM(operating_cost_inr) AS operating_cost,
           SUM(revenue_inr - operating_cost_inr) AS profit,
           ROUND(AVG(duration_minutes), 1) AS avg_duration_min
    FROM fleet_trips
    GROUP BY route_id, source, destination
)
SELECT *,
       RANK() OVER (ORDER BY profit DESC) AS profit_rank,
       ROUND(100.0 * profit / NULLIF(revenue, 0), 2) AS margin_pct
FROM route_stats
WHERE trips >= 10 -- ignore routes with too few trips to compare fairly
ORDER BY profit_rank
LIMIT 15;
```

Loss-making trips by route

```
SELECT route_id,
       COUNT(*) FILTER (WHERE revenue_inr < operating_cost_inr) AS loss_trips,
       COUNT(*) AS trips,
       ROUND(100.0 * COUNT(*) FILTER (WHERE revenue_inr < operating_cost_inr)
             / COUNT(*), 2) AS loss_trip_pct
FROM fleet_trips
GROUP BY route_id
HAVING COUNT(*) >= 10
ORDER BY loss_trip_pct DESC
LIMIT 10;
```

10.7 Cost composition and maintenance

The cost components are converted from columns to rows so that the share of each cost type can be computed in a single query. This is the same normalisation recommended in Section 9 for the cost composition visual.

```
WITH cost_long AS (
  SELECT vehicle_type, 'Fuel' AS cost_type, fuel_cost_inr AS amount FROM fleet_trips
  UNION ALL
  SELECT vehicle_type, 'Maintenance', maintenance_cost_inr FROM fleet_trips
  UNION ALL
  SELECT vehicle_type, 'Toll', toll_cost_inr FROM fleet_trips
  UNION ALL
  SELECT vehicle_type, 'Driver', driver_cost_inr FROM fleet_trips
)
SELECT vehicle_type, cost_type,
       SUM(amount) AS total_cost,
       ROUND(100.0 * SUM(amount) /
             SUM(SUM(amount)) OVER (PARTITION BY vehicle_type), 2) AS pct_of_vehicle_type_cost
FROM cost_long
GROUP BY vehicle_type, cost_type
ORDER BY vehicle_type, total_cost DESC;
```

Maintenance spend by maintenance type

```
SELECT maintenance_type,
       COUNT(*) AS records,
       SUM(maintenance_cost_inr) AS total_maintenance_cost,
       ROUND(AVG(maintenance_cost_inr), 2) AS avg_maintenance_cost,
       ROUND(100.0 * SUM(maintenance_cost_inr)
             / SUM(SUM(maintenance_cost_inr)) OVER (), 2) AS pct_of_total
FROM fleet_trips
GROUP BY maintenance_type
ORDER BY total_maintenance_cost DESC;
```

Top 10 vehicles by maintenance cost

```
SELECT vehicle_id, vehicle_model, vehicle_age_years,
       SUM(maintenance_cost_inr) AS total_maintenance_cost,
       COUNT(*) AS trips
FROM fleet_trips
GROUP BY vehicle_id, vehicle_model, vehicle_age_years
ORDER BY total_maintenance_cost DESC
LIMIT 10;
```

10.8 Trip status, ratings and drivers

```
-- Trip status distribution and monthly completion rate
SELECT trip_status, COUNT(*) AS trips,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*) OVER ()), 2) AS pct_of_trips
FROM fleet_trips
GROUP BY trip_status
ORDER BY trips DESC;

SELECT DATE_TRUNC('month', trip_date)::date AS month_start,
       ROUND(100.0 * COUNT(*) FILTER (WHERE trip_status = 'Completed')
             / COUNT(*), 2) AS completion_rate_pct
FROM fleet_trips
GROUP BY 1
ORDER BY 1;
```

Driver scorecard (minimum 20 trips)

```
SELECT driver_id, driver_name,
       COUNT(*) AS trips,
       ROUND(AVG(driver_rating), 2) AS avg_driver_rating,
       ROUND(AVG(customer_rating), 2) AS avg_customer_rating,
       ROUND(100.0 * COUNT(*) FILTER (WHERE trip_status = 'Completed')
            / COUNT(*), 2) AS completion_rate_pct,
       DENSE_RANK() OVER (ORDER BY AVG(driver_rating) DESC) AS rating_rank
FROM fleet_trips
GROUP BY driver_id, driver_name
HAVING COUNT(*) >= 20
ORDER BY rating_rank, trips DESC;
```

10.9 Outlier screening

Outliers are flagged for investigation, not deleted. The query marks trips whose cost per kilometre is more than three standard deviations from the mean.

```
WITH t AS (
    SELECT trip_id, vehicle_type, distance_km, operating_cost_inr,
           operating_cost_inr / NULLIF(distance_km, 0) AS cost_per_km
    FROM fleet_trips
    WHERE distance_km > 0
),
s AS (SELECT AVG(cost_per_km) AS mu, STDDEV(cost_per_km) AS sd FROM t)
SELECT t.*, ROUND((t.cost_per_km - s.mu) / NULLIF(s.sd, 0), 2) AS z_score
FROM t CROSS JOIN s
WHERE ABS((t.cost_per_km - s.mu) / NULLIF(s.sd, 0)) > 3
ORDER BY z_score DESC;
```

10.10 Reconciliation with Power BI

Run Query 10.3 and compare each result with the Power BI card for the same period and filters. Differences usually come from filters left on in the report, different handling of nulls or duplicates, or a different definition of operating cost.

KPI	SQL source	DAX measure	Match?
Total trips	COUNT(DISTINCT trip_id)	[Total Trips]	To be confirmed
Total revenue	SUM(revenue_inr)	[Total Revenue]	To be confirmed
Total operating cost	SUM(operating_cost_inr)	[Total Operating Cost]	To be confirmed
Profit margin %	(rev - cost) / rev	[Profit Margin %]	To be confirmed
Fuel efficiency	SUM(distance) / SUM(fuel)	[Fuel Efficiency KMPL]	To be confirmed
Completion rate %	Completed / all trips	[Trip Completion Rate %]	To be confirmed

Dialect notes

- **MySQL:** replace `DATE_TRUNC('month', d)` with `DATE_FORMAT(d, '%Y-%m-01')`; replace `COUNT(*) FILTER (WHERE ...)` with `SUM(CASE WHEN ... THEN 1 ELSE 0 END)`; window functions need MySQL 8.0 or later.
- **SQL Server:** replace `DATE_TRUNC` with `DATEFROMPARTS(YEAR(d), MONTH(d), 1)`; replace `LIMIT n` with `TOP n`; use the same CASE-based replacement for `FILTER`; use `STDEV` instead of `STDDEV`.
- **SQLite:** use `strftime('%Y-%m-01', d)` for month truncation and the CASE-based replacement for `FILTER`.

11. Python Analysis

Python is used for exploratory data analysis (EDA), statistical checks and chart generation. It complements the SQL layer by making outlier detection, correlation analysis and distribution plots easy to produce and document. The code below uses pandas, NumPy, matplotlib and seaborn and is designed to run in a Jupyter Notebook or as a script.

11.1 Setup and loading

```
# pip install pandas numpy matplotlib seaborn openpyxl
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

pd.set_option("display.max_columns", 50)
pd.set_option("display.float_format", lambda x: f"{x:,.2f}")
sns.set_theme(style="whitegrid")

OUT = Path("outputs"); OUT.mkdir(exist_ok=True)

df = pd.read_csv("fleet_trips.csv")          # or pd.read_excel("fleet_trips.xlsx")
print(df.shape)                             # expect 27 columns
df.info()
df.head()
```

11.2 Cleaning and type conversion

```
# Standardise column names
df.columns = (df.columns.str.strip().str.lower().str.replace(" ", "_"))

# Data types
df["trip_date"] = pd.to_datetime(df["trip_date"], errors="coerce")

num_cols = ["driver_rating", "distance_km", "duration_minutes", "fuel_consumed_liters",
            "fuel_cost_inr", "maintenance_cost_inr", "toll_cost_inr", "driver_cost_inr",
            "operating_cost_inr", "revenue_inr", "customer_rating", "vehicle_age_years"]
df[num_cols] = df[num_cols].apply(pd.to_numeric, errors="coerce")

# Clean text columns (trim spaces, consistent capitalisation)
text_cols = ["vehicle_type", "driver_name", "source", "destination", "payment_mode",
            "trip_status", "maintenance_type", "location", "vehicle_model", "fuel_type"]
for c in text_cols:
    df[c] = df[c].astype("string").str.strip().str.title()

rows_before = len(df)
# Duplicates: inspect first, then decide
dups = df[df.duplicated(subset="trip_id", keep=False)].sort_values("trip_id")
print("Duplicate trip_id rows:", len(dups))
df = df.drop_duplicates(subset="trip_id", keep="first")          # only if confirmed as errors
print("Rows before / after:", rows_before, "/", len(df))
```

11.3 Data quality report

```
quality = pd.DataFrame({
    "dtype":      df.dtypes.astype(str),
    "null_count": df.isna().sum(),
    "null_pct":   (df.isna().mean() * 100).round(2),
    "n_unique":   df.nunique(),
})
print(quality)

# Range checks
checks = {
    "negative_distance": (df["distance_km"] < 0).sum(),
    "negative_cost":      (df[["fuel_cost_inr", "maintenance_cost_inr", "toll_cost_inr",
                                "driver_cost_inr", "operating_cost_inr"]] < 0).any(axis=1).sum(),
    "negative_revenue":   (df["revenue_inr"] < 0).sum(),
    "driver_rating_oob":  (~df["driver_rating"].between(0, 5) & df["driver_rating"].notna()).sum(),
    "customer_rating_oob": (~df["customer_rating"].between(0, 5) & df["customer_rating"].notna()).sum(),
    "invalid_dates":      df["trip_date"].isna().sum(),
}
print(pd.Series(checks, name="issues"))

# Category consistency
for c in ["trip_status", "vehicle_type", "fuel_type", "payment_mode"]:
    print(df[c].value_counts(dropna=False), "\n")
```

11.4 Does Operating_Cost_INR include its components?

```
components = df[["fuel_cost_inr", "maintenance_cost_inr", "toll_cost_inr", "driver_cost_inr"]].sum(axis=1)
diff = df["operating_cost_inr"] - components

summary = pd.Series({
    "equals_sum": (diff.abs() < 1).mean(),
    "higher_than_sum": (diff >= 1).mean(),
    "lower_than_sum": (diff <= -1).mean(),
})
print((summary * 100).round(2).astype(str) + " % of trips")
```

If almost all rows equal the sum, use `Operating_Cost_INR` as the total cost. Otherwise set an explicit definition, for example `total_cost = components.sum(axis=1)`, and apply it identically in SQL and DAX.

11.5 Feature engineering

```
df["profit_inr"] = df["revenue_inr"] - df["operating_cost_inr"]
df["cost_per_km"] = np.where(df["distance_km"] > 0, df["operating_cost_inr"] / df["distance_km"], np.nan)
df["revenue_per_km"] = np.where(df["distance_km"] > 0, df["revenue_inr"] / df["distance_km"], np.nan)
df["kmpl"] = np.where(df["fuel_consumed_liters"] > 0, df["distance_km"] / df["fuel_consumed_liters"], np.nan)
df["duration_hours"] = df["duration_minutes"] / 60
df["is_completed"] = (df["trip_status"] == "Completed").astype(int)
df["is_loss_trip"] = (df["profit_inr"] < 0).astype(int)
df["month"] = df["trip_date"].dt.to_period("M").dt.to_timestamp()
df["weekday"] = df["trip_date"].dt.day_name()
df["age_band"] = pd.cut(df["vehicle_age_years"], bins=[-0.1, 2, 5, 8, np.inf],
                        labels=["0-2 yrs", "3-5 yrs", "6-8 yrs", "9+ yrs"])
```

11.6 Overall KPIs

```
total_rev, total_cost = df["revenue_inr"].sum(), df["operating_cost_inr"].sum()
kpis = {
    "Total trips": df["trip_id"].nunique(),
    "Total revenue (INR)": total_rev,
    "Total op. cost (INR)": total_cost,
    "Total profit (INR)": total_rev - total_cost,
    "Profit margin %": 100 * (total_rev - total_cost) / total_rev if total_rev else np.nan,
    "Total distance (km)": df["distance_km"].sum(),
    "Total fuel (L)": df["fuel_consumed_liters"].sum(),
    # weighted ratio, matches the DAX measure
    "Fuel efficiency (km/L)": df["distance_km"].sum() / df["fuel_consumed_liters"].sum(),
    "Completion rate %": 100 * df["is_completed"].mean(),
    "Avg driver rating": df["driver_rating"].mean(),
    "Avg customer rating": df["customer_rating"].mean(),
}
kpi_table = pd.Series(kpis, name="value")
print(kpi_table)
kpi_table.to_csv(OUT / "kpi_summary.csv")

# Note: the simple mean of per-trip kmpl is NOT the same as the weighted ratio above
print("Mean of per-trip km/L:", df["kmpl"].mean())
```

11.7 Group analysis

```
def summarise(g):
    return pd.Series({
        "trips": g["trip_id"].nunique(),
        "revenue": g["revenue_inr"].sum(),
        "operating_cost": g["operating_cost_inr"].sum(),
        "profit": g["profit_inr"].sum(),
        "distance_km": g["distance_km"].sum(),
        "kmpl": g["distance_km"].sum() / g["fuel_consumed_liters"].sum(),
        "cost_per_km": g["operating_cost_inr"].sum() / g["distance_km"].sum(),
        "margin_pct": 100 * g["profit_inr"].sum() / g["revenue_inr"].sum(),
        "completion_pct": 100 * g["is_completed"].mean(),
    })

by_vehicle = df.groupby("vehicle_type").apply(summarise).sort_values("revenue", ascending=False)
by_fuel = df.groupby("fuel_type").apply(summarise)
by_route = (df.groupby(["route_id", "source", "destination"]).apply(summarise)
            .query("trips >= 10").sort_values("profit", ascending=False))
by_month = df.groupby("month").apply(summarise)

print(by_vehicle)
print(by_route.head(10))      # most profitable routes
print(by_route.tail(10))     # least profitable routes

by_vehicle.to_csv(OUT / "by_vehicle_type.csv")
by_route.to_csv(OUT / "by_route.csv")
```

Maintenance and driver analysis

```
maint = (df.groupby("maintenance_type")["maintenance_cost_inr"]
        .agg(records="count", total="sum", average="mean")
        .assign(pct_of_total=lambda x: 100 * x["total"] / x["total"].sum())
        .sort_values("total", ascending=False))
print(maint)

drivers = (df.groupby(["driver_id", "driver_name"])
          .agg(trips=("trip_id", "nunique"),
              avg_driver_rating=("driver_rating", "mean"),
              avg_customer_rating=("customer_rating", "mean"),
              completion_pct=("is_completed", lambda s: 100 * s.mean()))
          .query("trips >= 20")
          .sort_values("avg_driver_rating", ascending=False))
print(drivers.head(10))
```

11.8 Relationships between variables

```
corr_cols = ["distance_km", "duration_minutes", "fuel_consumed_liters", "fuel_cost_inr",
             "maintenance_cost_inr", "toll_cost_inr", "driver_cost_inr",
             "operating_cost_inr", "revenue_inr", "vehicle_age_years",
             "driver_rating", "customer_rating"]
corr = df[corr_cols].corr(method="spearman") # rank-based, robust to outliers

plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0, square=True)
plt.title("Spearman correlation matrix")
plt.tight_layout(); plt.savefig(OUT / "correlation_heatmap.png", dpi=150); plt.close()

# Specific questions from Section 3
print("Distance vs revenue:      ", df["distance_km"].corr(df["revenue_inr"], method="spearman"))
print("Distance vs duration:    ", df["distance_km"].corr(df["duration_minutes"], method="spearman"))
print("Vehicle age vs maintenance:", df["vehicle_age_years"].corr(df["maintenance_cost_inr"], method="spearman"))
print("Driver vs customer rating: ", df["driver_rating"].corr(df["customer_rating"], method="spearman"))
```

11.9 Outlier detection (IQR method)

```
def iqr_outliers(s, k=1.5):
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return (s < q1 - k * iqr) | (s > q3 + k * iqr)

outlier_report = {}
for c in ["distance_km", "duration_minutes", "fuel_consumed_liters",
         "operating_cost_inr", "revenue_inr", "cost_per_km"]:
    flag = iqr_outliers(df[c].dropna())
    outlier_report[c] = int(flag.sum())
print(pd.Series(outlier_report, name="iqr_outliers"))

# Keep the flagged trips for review; do not drop them automatically
df["cost_outlier"] = iqr_outliers(df["cost_per_km"].fillna(df["cost_per_km"].median()))
df.loc[df["cost_outlier"],
       ["trip_id", "vehicle_type", "route_id", "distance_km", "operating_cost_inr", "cost_per_km"]
       ].sort_values("cost_per_km", ascending=False).to_csv(OUT / "cost_outliers_for_review.csv", index=False)
```

11.10 Visualisations

```
# 1. Monthly revenue vs operating cost
m = df.groupby("month")[["revenue_inr", "operating_cost_inr"]].sum()
ax = m.plot(figsize=(10, 4.5), marker="o")
ax.set_title("Monthly revenue vs operating cost"); ax.set_ylabel("INR"); ax.set_xlabel("")
plt.tight_layout(); plt.savefig(OUT / "monthly_revenue_vs_cost.png", dpi=150); plt.close()

# 2. Trips and fuel efficiency by vehicle type
fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
by_vehicle["trips"].plot.bar(ax=ax[0], color="#1F3A5F", title="Trips by vehicle type")
by_vehicle["kmpl"].plot.bar(ax=ax[1], color="#2E8B6F", title="Fuel efficiency (km/L)")
plt.tight_layout(); plt.savefig(OUT / "vehicle_type_trips_kmpl.png", dpi=150); plt.close()

# 3. Cost composition by vehicle type (100% stacked)
comp = (df.groupby("vehicle_type")[["fuel_cost_inr", "maintenance_cost_inr",
                                     "toll_cost_inr", "driver_cost_inr"]].sum())
(comp.div(comp.sum(axis=1), axis=0) * 100).plot.bar(stacked=True, figsize=(10, 5))
plt.ylabel("% of component cost"); plt.title("Cost composition by vehicle type")
plt.tight_layout(); plt.savefig(OUT / "cost_composition.png", dpi=150); plt.close()

# 4. Distance vs revenue scatter
plt.figure(figsize=(7, 5))
sns.scatterplot(data=df, x="distance_km", y="revenue_inr", hue="vehicle_type", alpha=0.5)
plt.title("Distance vs revenue"); plt.tight_layout()
plt.savefig(OUT / "distance_vs_revenue.png", dpi=150); plt.close()

# 5. Vehicle age vs maintenance cost
plt.figure(figsize=(7, 5))
sns.regplot(data=df, x="vehicle_age_years", y="maintenance_cost_inr",
            scatter_kws={"alpha": 0.3}, lowess=True)
plt.title("Vehicle age vs maintenance cost"); plt.tight_layout()
plt.savefig(OUT / "age_vs_maintenance.png", dpi=150); plt.close()

# 6. Rating distributions and status mix
fig, ax = plt.subplots(1, 3, figsize=(15, 4))
sns.histplot(df["driver_rating"].dropna(), bins=20, ax=ax[0]).set_title("Driver rating")
sns.histplot(df["customer_rating"].dropna(), bins=20, ax=ax[1]).set_title("Customer rating")
df["trip_status"].value_counts().plot.pie(ax=ax[2], autopct="%1.1f%%", ylabel="", title="Trip status")
plt.tight_layout(); plt.savefig(OUT / "ratings_and_status.png", dpi=150); plt.close()
```

11.11 Reconciliation with SQL and Power BI

```
# Compare Python KPIs with values exported from SQL (Query 10.3) and Power BI cards.
sql_kpis = pd.read_csv("sql_kpi_export.csv", index_col=0).squeeze("columns") # your SQL export

compare = pd.DataFrame({"python": kpi_table, "sql": sql_kpis})
compare["abs_diff"] = (compare["python"] - compare["sql"]).abs()
compare["match"] = compare["abs_diff"] < 0.01
print(compare)
```

Any mismatch should be traced to a specific cause (filters, duplicates, nulls, or definition of operating cost) and documented before the dashboard is shared.

11.12 Optional: running the SQL from Python

```
# pip install sqlalchemy psycopg2-binary
from sqlalchemy import create_engine
engine = create_engine("postgresql+psycopg2://user:password@localhost:5432/fleet_db")

df_sql = pd.read_sql(
    "SELECT vehicle_type, SUM(revenue_inr) AS revenue FROM fleet_trips GROUP BY vehicle_type",
    engine)
print(df_sql)
```

12. Analysis Workflow and Validation

Follow a repeatable workflow so the report can be maintained and trusted.

- Import source data and document its source, date range, and row count.
- Clean data in Power Query and record the main transformations.
- **Profile the data in SQL (Section 10.2) and Python (Section 11.3) and resolve data-quality issues before modeling.**
- Create dimensions and relationships; check for unmatched keys and unexpected many-to-many relationships.
- Build core DAX measures and verify them against simple totals calculated from the source data.
- **Run the SQL KPI query (10.3) and the Python KPI summary (11.6) and reconcile both with the DAX measures.**
- **Use the Python EDA (Sections 11.7 to 11.10) to find patterns, outliers and relationships worth showing in the dashboard.**
- Build report pages and add slicers for date, vehicle type, route, driver, location, and status as appropriate.
- Test whether filters update all intended visuals and whether date sorting is correct.
- Check blanks, zero denominators, negative or implausible values, currency formatting, and totals.
- Save the PBIX file and export a PDF snapshot for sharing.

Validation checklist

- Total trip count matches the agreed trip grain and unique Trip_ID logic.
- Revenue and cost totals reconcile to the source data.
- Profit calculation uses the agreed definition of operating cost.
- Trip completion rate uses the correct completed-status label and denominator.
- Fuel efficiency is not divided by zero and uses consistent units.
- Monthly visuals are sorted chronologically rather than alphabetically.
- Slicers and cross-highlighting behave as expected.
- **SQL, Python and DAX produce the same values for each KPI under the same filters.**
- **Fuel efficiency uses the weighted ratio (total distance / total fuel) in all three tools.**
- **Outliers flagged in SQL/Python have been reviewed and the decision to keep or exclude them is documented.**

13. Business Insights and Recommendations Framework

Use the finished report to write evidence-based findings. The statements below are prompts for analysis, not claims about the current dataset.

Area	What to investigate	Potential action if supported by evidence
Cost control	Which vehicle types, routes, or periods have the highest cost per trip or per kilometre? (SQL 10.5, Python 11.7)	Review route planning, idle time, maintenance practices, and cost allocation.

Area	What to investigate	Potential action if supported by evidence
Fuel efficiency	Which vehicle types have lower KM/L after accounting for comparable trip conditions?	Investigate vehicle condition, driving patterns, and route characteristics.
Maintenance	Which vehicles or maintenance categories contribute most to maintenance spending? (SQL 10.7)	Review preventive maintenance schedule and recurring repair causes.
Trip completion	Which dates, routes, or vehicle types have lower completion rates? (SQL 10.8)	Investigate operational causes and status recording quality.
Service quality	How do driver and customer ratings vary across categories?	Review low-rated journeys and identify coaching or service improvements.
Revenue performance	Which routes, vehicle types, or months have stronger revenue per kilometre?	Review utilization, demand, and pricing decisions.
Loss-making trips	Which routes have a high share of trips where revenue is below cost? (SQL 10.6)	Review pricing, route design and cost allocation for those routes.
Anomalies	Which trips show unusual cost per kilometre or duration? (SQL 10.9, Python 11.9)	Check data entry first, then investigate operational causes.

When presenting results, include the actual value, comparison period/category, and relevant filter context. Avoid claiming that one factor caused another unless the analysis supports causality.

14. Testing, Limitations, and Future Enhancements

Known limitations to document

- Map accuracy depends on location quality, geographic context, and available coordinates.
- A source-to-destination route line generally needs a suitable route visual or geographic coordinates; a basic map does not automatically draw route paths.
- A dashboard is only as reliable as the data quality and definitions used.
- Ratings may be missing or biased toward users who submitted feedback.
- Profit and cost metrics depend on whether the supplied operating-cost field includes all expense components.
- No numeric findings should be reported until measures are validated against the actual data.
- SQL syntax differs slightly between database systems; the queries in Section 10 target PostgreSQL.
- The Python analysis is descriptive. Correlations and group differences do not establish causation, and small groups (few trips) can produce unstable averages.

Possible future enhancements

- Add vehicle utilization rate if active/available time or capacity data is available.
- Add a maintenance schedule and overdue-service indicator if maintenance dates are available.
- Add revenue and cost targets for KPI/gauge comparisons.
- Add drill-through pages for individual vehicles, drivers, and routes.
- Publish to Power BI Service with scheduled refresh and role-level security if required.
- Connect Power BI directly to the SQL database so that the dashboard and the SQL analysis share one source of truth.
- Automate the Python EDA as a scheduled script that exports a refreshed KPI and outlier report.
- Add forecasting only after sufficient time-series history and validation.

15. Project Conclusion

The Transportation & Fleet Operations Analytics Dashboard demonstrates an end-to-end analytics workflow: data preparation, dimensional modeling, DAX measure development, SQL analysis, Python exploratory analysis, interactive visualization, and validation. When populated with validated results, it can provide a consistent view of transportation activity, financial performance, fleet efficiency, maintenance spending, and service indicators.

16. Resume Description and Interview Explanation

Resume project description

Developed an end-to-end analytics project on transportation and fleet operations data. Cleaned and modeled trip-level data with Power Query and a star schema, wrote SQL queries (aggregations, window functions, CTEs) for data-quality checks, KPI calculation and route/vehicle analysis, performed exploratory analysis in Python (pandas, matplotlib, seaborn) including outlier detection and correlation analysis, and built an interactive Power BI dashboard with DAX measures for revenue, operating cost, profit, fuel efficiency, maintenance expense and driver ratings. Reconciled KPIs across SQL, Python and DAX to validate results.

Short interview explanation

My project is a Transportation & Fleet Operations Analytics Dashboard. I started by profiling and cleaning trip-level data, checking duplicates, nulls and whether the operating-cost field already included its components. I used SQL for KPI queries, window functions for monthly trends and route rankings, and CTEs for cost composition. In Python I did exploratory analysis with pandas, flagged outliers using the IQR method and examined correlations. I then built a star-schema model and DAX measures in Power BI with pages for executive KPIs, fleet performance, route analysis, maintenance costs and driver/customer ratings. Before presenting anything I reconciled totals between SQL, Python and Power BI.